

(2026. 06. 01.)

Project Title: Ouroboros

Overview: This is a grid-based puzzle game implemented in F#, which is a combination of sokoban and snake game genres. The player is supposed to move the snake to form shapes including ouroboros a single ouroboros (see [Explanation: The Goal of the Game](#)).

Requirements:

1. Game Entities

The world is a sokoban style 2D grid. Every cell is either empty, a wall, or a node. Each wall and node occupy a 1x1 cell. The map is surrounded by walls. The size of a map is usually 10x10 or something similar.

- Chain: A sequence of adjacent nodes where each adjacent nodes are linked by a 'bite'. To intuitively grasp these notions, it seems better to think of a node as a snake head. So, a chain is intuitively a snake, consisting of snake heads only, connected by bites.
- The Glowing Nodes: Some of the nodes are glowing. These glowings indicate that those nodes are currently under player control. The player can move these glowing nodes simultaneously using keyboard input. (See [Section 3: Input and Movement](#).)
- Nodes & Sides: Each node has four sides (since the map is grid-based). There are only two types:
 - Soft Side (= face-side = eye-side) of a node: So, there's always exactly one soft side for each node. It corresponds to the side where face/mouth/eyes of the snake head locate. A soft side exerts zero force.
 - Rigid Sides of a Node: The other three sides of the node that are not a soft side. These sides may transmit force and velocity when the node is moving.

The game mechanics itself is quite intuitive. However, it is not that easy to explain it in words.

2. Loop

A single input triggers a three-phase process: I. Pre-Action, II. Action, III. Glow Transfer. The cycle repeats forever until the level ends. (See [Section 5: Game Interface and Graphics](#) for level ending) These three phases occur only at the algorithmic level. The player merely sees the final result of the process (the cycle), so it is hard to observe the internal process directly through the game interface only. (A minor change has been made: see [Explanation: Visualization](#).) On one hand, I tried to explain the basic idea of how those rules and mechanics are going to be implemented for the sole purpose of accurately describing the rules and game mechanics. On the other hand, it's simply enough to focus on the behavioral aspects when testing this game.

I. Pre-Action

1. Initialization: Every node is given with a velocity (to be precise, a velocity in this proposal is either the zero vector or one of the north, east, south, west directions.) based on the input. (See [Section 3: Input and Movement](#).)
2. Propagation: Implementation of the velocity transfers across rigid ↔ rigid contacts. In the most simple language, this means that if a box is being pushed and if the box pushes another box then the two boxes move together in the same velocity.
3. Conflict Check: If any rigid side of a node with non-zero velocity comes into contact with a wall or a node with zero velocity, that entire rigid structure is blocked (= zero velocity).

II. Action

- Face-to-face: If a node's face (soft side) tries to push another node's face, it repulses back; the entire bodies remain at the initial position, with the updated velocity of zero.
- Bite: If a soft (rigid) side attempts to push (in the velocity that was computed in the pre-action phase) a rigid (soft) side of another node, a bite occurs if their current velocities do not match. (See [Section 4: The Bite Mechanics](#) and [Explanation: An Exception to Rules](#).) After biting, the velocities of the resulting chains become zero.

The program performs these actions until no more of them are available. After that, the following actions occur for each of the remaining nodes.

1. Move: The node moves a single step according to the current velocity.
2. ~~Exception to 1: Forward move.~~ (See [Section 3: Input and Movement](#) and [Explanation: An Exception to Rules](#).)

III. Glow Transfer

The glow transfer occurs after the first and second phases are terminated. See also [Section 4: The Bite Mechanics](#).

3. Input and Movement

The results of an input (WASD or Arrow Keys) depend on each glowing head's current facing.

Input vs. Facing	Action	Explanation
Orthogonal	Rotation	Head rotates freely (without moving forward/backward). If it faces its own 'neck', it detaches as a 1-node chain.
Same Direction	Forward Move	Snake-style: Head moves forward; body follows the path. (Only the head and the tail nodes bear nonzero velocity) (See Explanation: An Exception to Rules)
Opposite Direction	Backward Move	Rigid-body: The whole chain moves as one rigid body. (All nodes bear the input velocity.)

An **ouroboros** is made when a closed loop is created and remains after the action phase. Once a chain becomes a loop, it no longer has heads. To keep the player in control:

1. Uniform Glow: All nodes in the loop become glowing if the loop contains at least one glowing node.
2. Rigid Control: This loop ignores forward/backward moves. Any directional input assigns the entire loop with that velocity vector. The loop moves as a single, rigid body.
3. Splitting: If a foreign head bites the ouroboros, the loop splits. The same biting rule applies here. It becomes one or more chains, and the glow-transfer rules re-apply to determine the new player heads.
4. Glow Transfer: It is no different from ordinary chains (except that every node of the loop might be glowing).

4. The Bite Mechanics

A bite triggers if a head with (non-zero) velocity v touches and presses its soft (rigid) side against a rigid (soft) side of another node with velocity u , where $u \neq v$ (See [Explanation: An Exception to Rules](#)). As we mentioned in Sec. 1, it is useful to think each node as a snake head.

Bite Effects

- Splitting: If the target node is already being bitten by another biter node, the (old) biter node 'un-bites' their target node right before the new one bites it; therefore, the target chain splits into two chains in this case.
- Merging: The (new) biter node and the target node become internally connected. If the target was a mid-node of a chain, that chain splits, as we said; one half (containing the head node) fuses to the biter, the other (containing the end node) becomes a new independent chain. If the target was the end-node of a chain, a single long chain forms.
- Glow Transfer: The glow appears on the front-most node of the chain (the one in the chain that isn't biting any node), and, after the rearrangement of the glow, the player gains new control over the new chain.

5. Game Interface and Graphics

The player wins when an ouroboros is made and no other nodes (remnants) exist on the grid. ~~(The winning condition may be changed later if I find it impossible to design levels.)~~ (See [Explanation: The Goal of the Game](#).) We are going to use Raylib-cs for graphics (unless any kind of unexpected, unsolvable error occurs). The map, the game entities, plus the bodies, eyes, and bites should be represented (either at an abstract level or at a graphically realistic level) using the same graphic library, and they should be clearly visible. The player can select levels at the menu. WASD or Arrow Keys to navigate through levels, Q to quit the current level, R to restart the current level. (More keys are added; see [Explanation: More Keys](#))

Example Interaction: The player moves the snake. Initially, the snake is a single node, but after collecting 7 more nodes, the player eventually creates an ouroboros to win a level.

Explanation

The Goal of the Game

I previously wrote that the winning condition might change if I find it impossible to design levels, but the goal has *not* really changed. To be more precise, the player wins *a level* when a *single* ouroboros is made and no other nodes (remnants) exist on the grid.

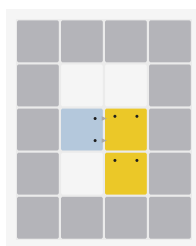
The below are three changes that were made after the proposal.

Visualization

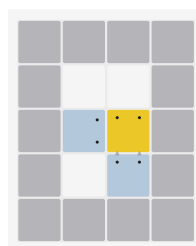
I previously wrote that the internal processes are not shown. However, the processes are shown in simple animation in the actual implementation. This change is quite optional, and it does not really change how the game works. But it makes the game more intuitive and user-friendly, making it easy for the players to play and test the game.

An Exception to Rules

In [Section 4: The Bite Mechanics](#), what I previously wrote is that a bite occurs (either A bites B or B bites A) if and only if a node A with non-zero velocity v presses its soft (rigid) side against a rigid (soft) side of another node B with velocity u , where $u \neq v$. However, there actually is a rare exception to this rule.



Before pressing Up



After pressing Up (the intended behavior)

When the player presses Up, both of the glowing nodes (the yellow ones) are assigned the upward direction. According to the previous rule, no bite should occur, since $\text{Up} = \text{Up}$. In this case, two nodes would overlap (the blue one in the first image would move right while the bottom one would move upward). To prevent this, we add an exception that a bite should also occur (as in the second image) when this kind of overlaps can happen.

(The actual implementation is slightly different from this exception rule, but functionally they are the same.)

(Now that we have this exception, we can safely ignore the sentences “Exception to 1: Forward move.” and “Only the head and the tail nodes bear nonzero velocity” that I previously wrote. Instead, each node is given a velocity according to the direction it is currently facing (for each snake-style forward move), which seems far more natural than the earlier rules.)

More Keys

Press Space/Enter to play a level, Z to undo, Esc to quit the game. These keys are more or less essential to the gameplay. The addition of the undo button was particularly inevitable, because some of the levels would be extremely frustrating to solve without the undo button. (Try it!)